

Constraint-Based Problem Solving

Exam Timetable Scheduling as a Constraint Satisfaction Problem

1. Scenario

A university must schedule the final examinations of five courses into three exam slots (one exam per day over three days). Two courses that share students cannot be placed in the same slot. The courses are: A – Artificial Intelligence, B – Database Systems, C – Computer Networks, D – Operating Systems, E – Software Engineering.

2. Conflicting pairs (must receive different slots)

No.	Pair	Meaning
1	A – B	AI and DBMS share students
2	A – C	AI and Networks share students
3	B – C	DBMS and Networks share students
4	B – D	DBMS and OS share students
5	C – D	Networks and OS share students
6	C – E	Networks and Software Engineering share students
7	D – E	OS and Software Engineering share students

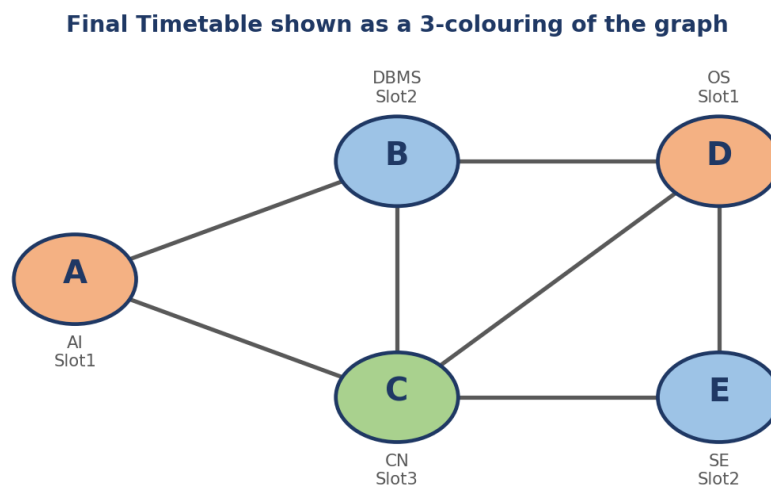


Figure 1: Final timetable displayed as a colouring of the constraint graph (same colour = same slot).

Part (a) – PEAS Description

a.1 PEAS Need

PEAS (Performance measure, Environment, Actuators, Sensors) is the standard framework for specifying the task environment of an intelligent agent before its design is attempted. Defining PEAS first makes it clear what “success” means, what the agent can sense, and what it is able to change. Here the agent is an automated exam-scheduling agent whose job is to build a conflict-free timetable.

a.2 PEAS table

PEAS element	Description for the exam-scheduling agent	Justification
Performance measure	• Number of violated constraints = 0 (hard requirement) • All 5 courses assigned to exactly one slot • Only the 3 available slots used • Computation cost (assignment attempts / backtracks) kept low • Optional: balanced load across slots	A timetable is only useful if no student has two exams in the same slot, so conflict-freeness is the primary measure. Search effort measures the efficiency of the agent.
Environment	• 5 courses (A–E) and 3 exam slots (Slot1–Slot3) • Student-enrolment data defining the 7 conflicting pairs • The current (partial) timetable • University exam rules (one exam per course per slot)	These are all the external objects and rules the agent must reason about; they do not change while the agent plans.
Actuators	• Assign a slot to a course • Un-assign / change a slot (backtracking) • Prune values from domains (internal reasoning action) • Publish / display the final timetable to students and staff	Actuators are what the agent can do to change the state of the timetable and communicate the result.
Sensors	• Reads the course list and slot list • Reads the conflict list from the registration database • Observes the current partial assignment • Detects constraint violations or empty domains	The agent needs the conflict data and its own current state to decide which assignment is legal.

a.3 Task-environment properties

Property	Value	Justification
Observability	Fully observable	All courses, slots and conflicts are known to the agent at the start.
Agents	Single agent	Only the scheduler acts; no competing agents.
Determinism	Deterministic	Assigning a slot always has the same effect on the timetable.
Episodic / Sequential	Sequential	Each assignment constrains later assignments (e.g. A=Slot1 forbids Slot1 for B and C).
Static / Dynamic	Static	Enrolment data does not change while the timetable is being computed.
Discrete / Continuous	Discrete	Finite variables (5) and finite values (3 slots).
Known / Unknown	Known	The rules (constraints) are given completely.

a.4 Justification of the PEAS design

- Performance measure is stated as hard constraints (zero conflicts) because a single clash makes the timetable invalid; efficiency is a secondary measure that is used later in Part (f).
- Environment is limited to what affects the decision, which keeps the model small and lets it be written as a CSP in Part (c).
- Actuators include un-assigning because backtracking search must be able to undo earlier decisions.
- Sensors include detection of empty domains because forward checking (Part e) uses this signal to detect dead ends early.

Part (b) – Agent Type Classification

b.1 Candidate agent types compared

Agent type	How it decides	Suitable here?	Reason
Simple reflex	Condition–action rules on the current percept only	No	A conflict-free timetable depends on all earlier assignments, not just the current percept. Fixed rules cannot handle every combination.
Model-based reflex	Keeps internal state/model, still uses rules	Partly	Tracking the partial timetable is needed, but there is no explicit goal or look-ahead, so it cannot plan around dead ends.
Goal-based	Uses a model + explicit goal + search/planning	Yes – best	The task has a clear goal (all constraints satisfied) and requires searching through assignments.
Utility-based	Maximises a utility function over outcomes	Not required	Every valid timetable is equally acceptable; there is no numeric preference between solutions.

b.2 Selected agent type: Goal-based agent

The exam scheduler is best classified as a goal-based agent that maintains an internal model of the partially built timetable and searches (backtracking / forward checking) until the goal test “all 7 constraints satisfied and all courses assigned” is true.

b.3 Justification points

1. Explicit goal. The goal state is well defined: every course has a slot and no conflicting pair shares a slot. A goal-based agent is built around exactly this kind of goal test.
2. Need for look-ahead / search. Choosing Slot1 for A affects B, C and, indirectly, D and E. The agent must consider the consequences of an action before committing, which is what search provides.
3. Internal model. The agent stores the current partial assignment and the conflict graph, so it knows what is legal. This is the “model” part used by goal-based agents.
4. Flexibility. If a constraint changes (e.g. a new conflict D–A), the goal and model are updated and the same search runs again. A reflex agent would need all of its rules rewritten.
5. A utility-based agent would be required only if there were quality preferences (e.g. minimise the number of students having exams on consecutive days). The question only asks for a conflict-free timetable, which is a yes/no goal.

b.4 Agent program (pseudo-code)

```
function EXAM-SCHEDULING-AGENT(percept) returns an action
  persistent: model <- (courses, slots, conflict graph)
               state <- current partial timetable
```

```
goal <- "all courses assigned, no conflicting pair in same slot"
state <- UPDATE-STATE(state, percept, model)
if GOAL-TEST(state) then return PUBLISH(state)
plan <- CSP-SEARCH(state, model)      # backtracking / forward checking
if plan = failure then return REPORT("no feasible timetable")
return FIRST-ACTION(plan)            # e.g. ASSIGN(A, Slot1)
```

Part (c) – CSP Formulation and Constraint Graph

c.1 Definition of a CSP

A Constraint Satisfaction Problem is defined by a triple (X, D, C) : a set of variables X , a domain D of allowed values for each variable, and a set of constraints C that restrict which combinations of values are allowed. A solution is a complete and consistent assignment, i.e. every variable has a value and no constraint is violated.

c.2 Variables (X)

Variable	Course	Meaning
A	Artificial Intelligence	Exam slot assigned to AI
B	Database Systems	Exam slot assigned to DBMS
C	Computer Networks	Exam slot assigned to CN
D	Operating Systems	Exam slot assigned to OS
E	Software Engineering	Exam slot assigned to SE

$X = \{A, B, C, D, E\}$. Justification: each course needs exactly one slot, so one variable per course is the natural choice.

c.3 Domains (D)

Variable	Domain
A	{Slot1, Slot2, Slot3}
B	{Slot1, Slot2, Slot3}
C	{Slot1, Slot2, Slot3}
D	{Slot1, Slot2, Slot3}
E	{Slot1, Slot2, Slot3}

Justification: the university has exactly three exam slots, and no course is restricted to particular slots, so all domains are identical with size 3. The unconstrained search space contains $3^5 = 243$ complete assignments.

c.4 Constraints (C)

Every conflicting pair gives one **binary inequality constraint** (the two courses must differ in slot):

ID	Constraint	Allowed value pairs	Reason
C1	$A \neq B$	6 of 9 pairs	Shared students in AI and DBMS

ID	Constraint	Allowed value pairs	Reason
C2	$A \neq C$	6 of 9 pairs	Shared students in AI and CN
C3	$B \neq C$	6 of 9 pairs	Shared students in DBMS and CN
C4	$B \neq D$	6 of 9 pairs	Shared students in DBMS and OS
C5	$C \neq D$	6 of 9 pairs	Shared students in CN and OS
C6	$C \neq E$	6 of 9 pairs	Shared students in CN and SE
C7	$D \neq E$	6 of 9 pairs	Shared students in OS and SE

Pairs that are not listed (A–D, A–E, B–E) have no constraint, so these courses may share a slot. All constraints are binary, so the problem is a binary CSP and can be drawn as a graph.

c.5 Constraint graph

In a constraint graph each variable is a vertex and each binary constraint is an edge joining the two variables.

Item	Content
Vertices (5)	A, B, C, D, E
Edges (7)	(A,B), (A,C), (B,C), (B,D), (C,D), (C,E), (D,E)

Vertex	Neighbours (adjacency list)	Degree
A	B, C	2
B	A, C, D	3
C	A, B, D, E	4
D	B, C, E	3
E	C, D	2

Constraint Graph (7 edges, 5 vertices)

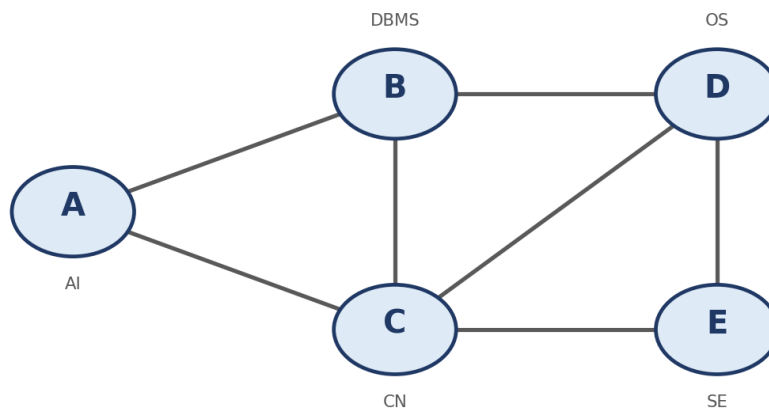


Figure 2: Constraint graph of the exam-scheduling CSP.

c.6 Structural observations (justification and insight)

- Total degree = $14 = 2 \times 7$ edges, which confirms that the adjacency list matches the edge list.
- C has the highest degree (4), so it is the most constrained variable. Choosing it early (an MRV/degree heuristic) would reduce search in general, but the question requires the fixed order $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$.
- The graph is a chain of three triangles: $\{A,B,C\}$, $\{B,C,D\}$ and $\{C,D,E\}$. Because each triangle needs 3 different slots, at least 3 slots are needed, and we have exactly 3, so the CSP is solvable only if the 3-colouring is tight.
- Once A, B, C take three different slots, D must equal A's slot (D differs from B and C) and E must equal B's slot (E differs from C and D). Hence there are exactly $3! = 6$ valid timetables, one for each ordering of the three slots over A, B, C. The search in Parts (d) and (e) finds the first of these.

c.7 Python representation of the CSP

```
VARIABLES = ["A", "B", "C", "D", "E"]
DOMAIN    = ["Slot1", "Slot2", "Slot3"]
EDGES     = [("A","B"),("A","C"),("B","C"),("B","D"),
              ("C","D"),("C","E"),("D","E")]    # constraints: u != v
NEIGHBOURS = {v: set() for v in VARIABLES}
for u, v in EDGES:
    NEIGHBOURS[u].add(v); NEIGHBOURS[v].add(u)
```

Part (d) – Plain Backtracking Search

d.1 Method and assumptions

- Backtracking search is a depth-first search that assigns one variable at a time and checks each new assignment only against variables that are already assigned. If no value works for a variable, the algorithm undoes the previous assignment (backtracks) and tries its next value.
- **Variable order (static):** $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$. **Value order:** Slot1 $\rightarrow$ Slot2 $\rightarrow$ Slot3 (written S1, S2, S3 in the tables).
- **Counting rules used:** an assignment attempt is every time a value is tried for a variable; a rejected attempt is an attempt that violates a constraint; a backtrack is the undoing of an earlier accepted assignment because a later variable has no legal value left.

d.2 Algorithm

```
function BACKTRACKING-SEARCH(csp) return RECURSIVE-BT({}, csp)
```

```
function RECURSIVE-BT(assignment, csp)
  if assignment is complete then return assignment
  var <- next unassigned variable in order A,B,C,D,E
  for each value in [Slot1, Slot2, Slot3] do
    if value is consistent with assignment (no assigned neighbour has it) then
      add {var = value} to assignment
      result <- RECURSIVE-BT(assignment, csp)
      if result != failure then return result
      remove {var = value} from assignment      # BACKTRACK
  return failure
```

d.3 Flowchart of plain backtracking

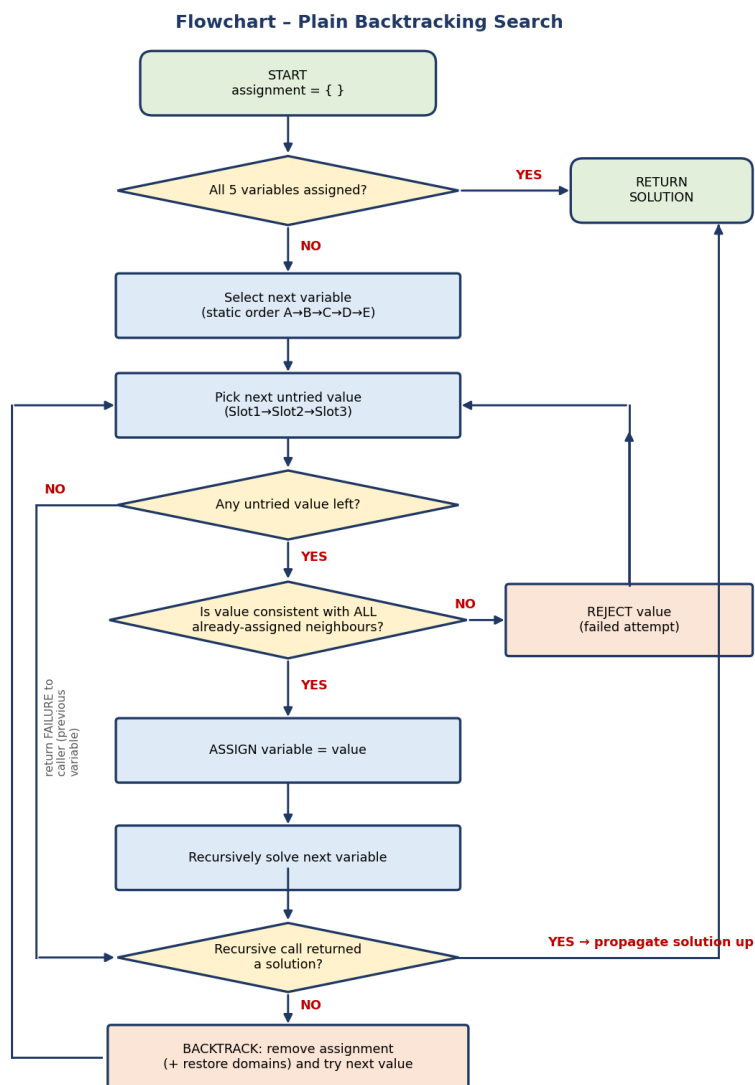


Figure 3: Flowchart of plain backtracking search.

d.4 Step-by-step table of every assignment attempt

Step	Var	Value tried	Consistency check against assigned neighbours	Result	Partial assignment after step
1	A	Slot1	No assigned neighbours	Accepted	{A=S1}
2	B	Slot1	A=S1 → same slot (A≠B violated)	Rejected	{A=S1}
3	B	Slot2	A=S1 ≠ S2 ✓	Accepted	{A=S1, B=S2}
4	C	Slot1	A=S1 → same slot (A≠C violated)	Rejected	{A=S1, B=S2}
5	C	Slot2	B=S2 → same slot (B≠C violated)	Rejected	{A=S1, B=S2}
6	C	Slot3	A=S1 ✓, B=S2 ✓	Accepted	{A=S1, B=S2, C=S3}

Step	Var	Value tried	Consistency check against assigned neighbours	Result	Partial assignment after step
7	D	Slot1	B=S2 ✓, C=S3 ✓ (E not yet assigned)	Accepted	{A=S1, B=S2, C=S3, D=S1}
8	E	Slot1	C=S3 ✓, D=S1 → same slot (D≠E violated)	Rejected	{A=S1, B=S2, C=S3, D=S1}
9	E	Slot2	C=S3 ✓, D=S1 ✓	Accepted	{A=S1, B=S2, C=S3, D=S1, E=S2} → COMPLETE

d.5 Explanation of the key steps

- Step 2 (B=S1 rejected). B is adjacent to A and A already holds Slot1, so Slot1 is illegal. The algorithm simply tries the next value, Slot2 (Step 3).
- Steps 4–5 (C=S1 and C=S2 rejected). C is adjacent to both A (S1) and B (S2), so only Slot3 survives. Two attempts were wasted before reaching it. This is the biggest saving available to forward checking in Part (e).
- Step 7 (D=S1 accepted). D is adjacent to B (S2) and C (S3) but not to A, so Slot1 is allowed even though A uses it.
- Step 8 (E=S1 rejected). E is adjacent to D (S1), so Slot1 is illegal; Slot2 is legal because C=S3 and D=S1 (Step 9).
- Backtracking steps. Every variable found a legal value before its domain was exhausted, so no earlier assignment ever had to be undone: 0 backtracks. The four rejected attempts are local retries within one variable.

d.6 Search tree

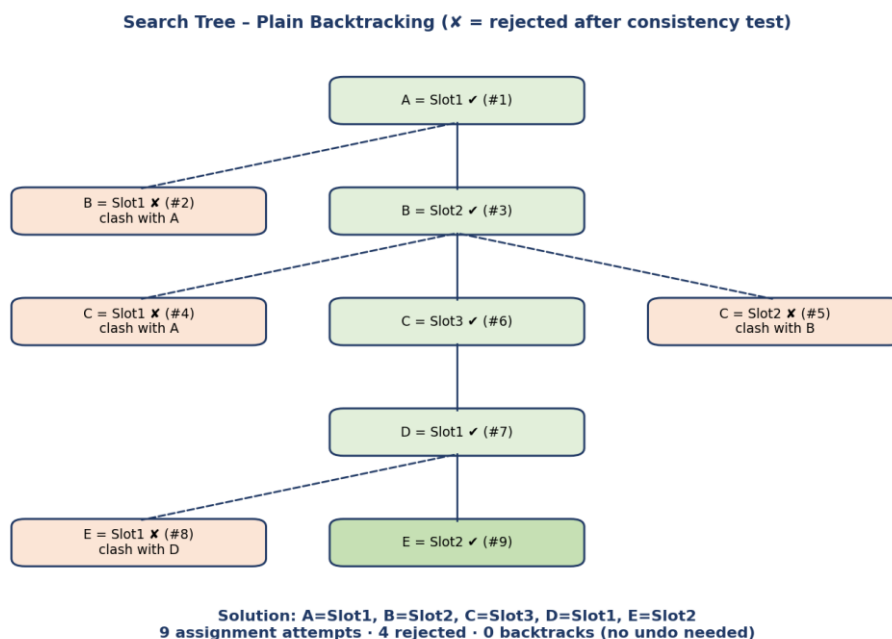


Figure 4: Search tree for plain backtracking. Numbers in brackets are the step numbers from the table.

d.7 Final valid timetable

Exam slot	Course(s) scheduled	Day
Slot1	A – Artificial Intelligence, D – Operating Systems	Day 1
Slot2	B – Database Systems, E – Software Engineering	Day 2
Slot3	C – Computer Networks	Day 3

Assignment: A = Slot1, B = Slot2, C = Slot3, D = Slot1, E = Slot2.

d.8 Verification of the solution against every constraint

Constraint	Values	Satisfied?
$A \neq B$	Slot1 $\neq$ Slot2	✓ Yes
$A \neq C$	Slot1 $\neq$ Slot3	✓ Yes
$B \neq C$	Slot2 $\neq$ Slot3	✓ Yes
$B \neq D$	Slot2 $\neq$ Slot1	✓ Yes
$C \neq D$	Slot3 $\neq$ Slot1	✓ Yes
$C \neq E$	Slot3 $\neq$ Slot2	✓ Yes
$D \neq E$	Slot1 $\neq$ Slot2	✓ Yes

All 7 constraints hold and all 5 courses are assigned, so the assignment is complete and consistent. Pairs A–D and A–E (same slot for A and D) are allowed because they are not conflicting pairs.

d.9 Result summary

Metric	Value
Assignment attempts	9
Accepted attempts	5
Rejected attempts	4
Backtracks (undoing an earlier assignment)	0

Part (e) – Backtracking with Forward Checking

e.1 Idea of forward checking

Forward checking (FC) is backtracking plus a look-ahead step. Whenever a variable X is assigned a value v , the algorithm looks at every unassigned neighbour Y of X and deletes v from Y 's domain. If any domain becomes empty (a wipe-out), the current value is rejected immediately, before any deeper search. Values that have already been deleted are never tried later.

e.2 Algorithm

```
function RECURSIVE-FC(assignment, domains)
  if assignment is complete then return assignment
  var <- next unassigned variable in order A,B,C,D,E
  for each value in domains[var] (in order Slot1,Slot2,Slot3) do
    new_domains <- copy of domains; new_domains[var] <- {value}
    for each unassigned neighbour Y of var do
      remove value from new_domains[Y]          # PRUNING
      if new_domains[Y] is empty then reject value; continue # WIPE-OUT
    add {var = value} to assignment
    result <- RECURSIVE-FC(assignment, new_domains)
    if result != failure then return result
    remove {var = value} from assignment        # BACKTRACK
  return failure
```

e.3 Flowchart of backtracking with forward checking

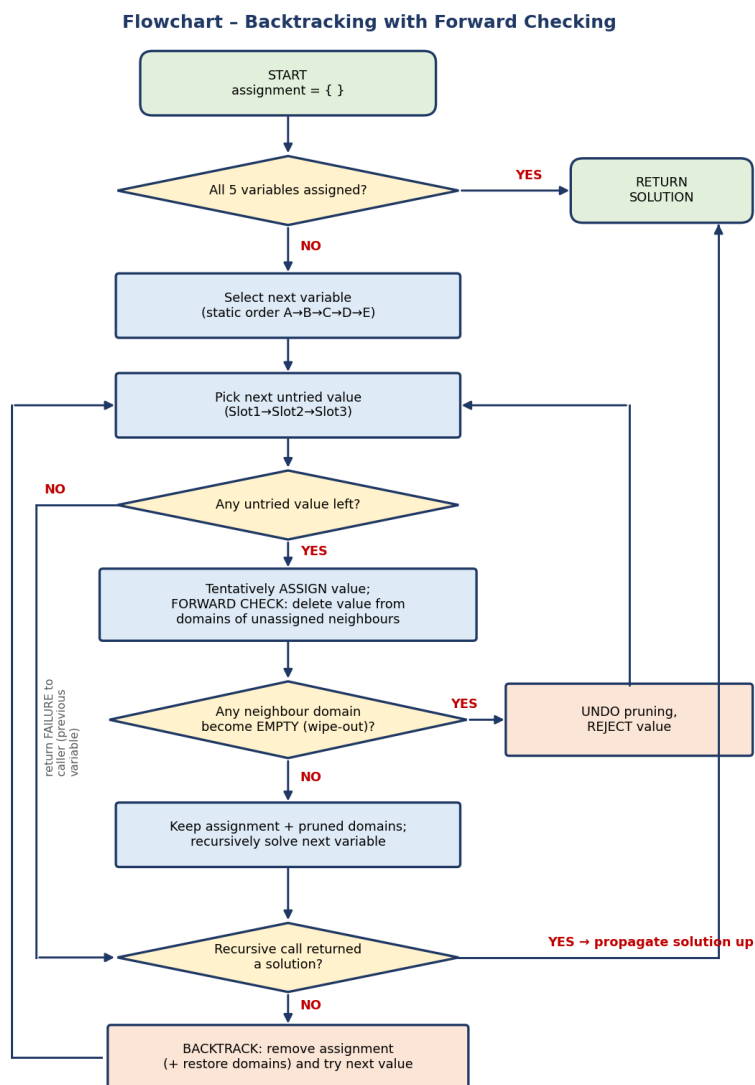


Figure 5: Flowchart of backtracking with forward checking (extra pruning and wipe-out test).

e.4 Initial domains

Variable	A	B	C	D	E
Domain	{1,2,3}	{1,2,3}	{1,2,3}	{1,2,3}	{1,2,3}

In the tables below 1, 2, 3 denote Slot1, Slot2, Slot3. Pruned values are shown in the “Removed” column.

e.5 Step-by-step assignments with domain pruning

Step	Assignment	Neighbours checked (unassigned)	Values removed	Domains after pruning	Wipe-out?
1	A = Slot1	B, C (D, E are not adjacent to A)	Slot1 from B Slot1 from C	A: {1} B: {2,3} C: {2,3} D: {1,2,3} E: {1,2,3}	No

Step	Assignment	Neighbours checked (unassigned)	Values removed	Domains after pruning	Wipe-out?
2	B = Slot2 (Slot1 already pruned, so first value tried is Slot2)	C, D (A already assigned)	Slot2 from C Slot2 from D	A: {1} B: {2} C: {3} D: {1,3} E: {1,2,3}	No
3	C = Slot3 (only value left in C's domain)	D, E (A, B already assigned)	Slot3 from D Slot3 from E	A: {1} B: {2} C: {3} D: {1} E: {1,2}	No
4	D = Slot1 (only value left in D's domain)	E (B, C already assigned)	Slot1 from E	A: {1} B: {2} C: {3} D: {1} E: {2}	No
5	E = Slot2 (only value left in E's domain)	None (all neighbours assigned)	–	All variables assigned	No

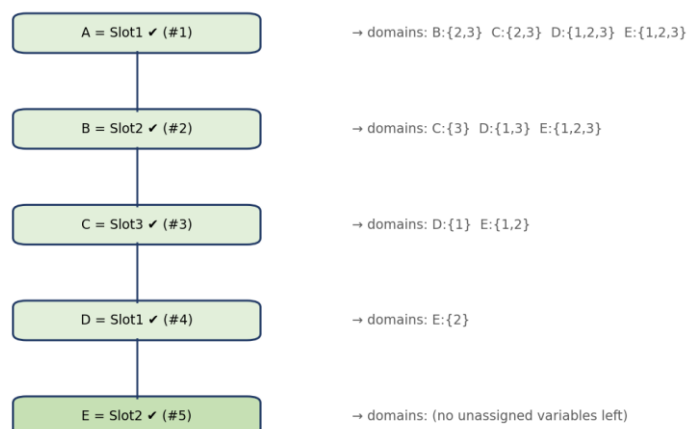
e.6 Domain evolution summary

Stage	A	B	C	D	E
Initial	{1,2,3}	{1,2,3}	{1,2,3}	{1,2,3}	{1,2,3}
After A = 1	{1}	{2,3}	{2,3}	{1,2,3}	{1,2,3}
After B = 2	{1}	{2}	{3}	{1,3}	{1,2,3}
After C = 3	{1}	{2}	{3}	{1}	{1,2}
After D = 1	{1}	{2}	{3}	{1}	{2}
After E = 2	{1}	{2}	{3}	{1}	{2}

Green = variable just assigned; yellow = domains reduced by forward checking in that step. In total 2 + 2 + 2 + 1 = 7 values were pruned.

e.7 Search tree

Search Tree - Backtracking with Forward Checking (pruned domains beside each node)



Solution: A=Slot1, B=Slot2, C=Slot3, D=Slot1, E=Slot2
5 assignment attempts · 0 rejected · 0 backtracks

Figure 6: Search tree for forward checking with the pruned domains after each assignment.

e.8 Explanation and justification

- After A = Slot1, B and C lose Slot1 because they are adjacent to A. The failed attempt B=Slot1 of the plain search never happens.
- After B = Slot2, C is left with a single value {Slot3}. Both wasted attempts C=Slot1 and C=Slot2 (Steps 4–5 of Part d) disappear.
- After C = Slot3, D's domain becomes {Slot1} and E's becomes {Slot1, Slot2}; no domain is empty, so no dead end is detected.
- After D = Slot1, E is reduced to {Slot2}. The failed attempt E=Slot1 of Part (d) is avoided, and E=Slot2 is assigned directly.
- At every step every unassigned domain stayed non-empty, so there was no wipe-out and 0 backtracks.

e.9 Final valid timetable

Exam slot	Course(s) scheduled	Day
Slot1	A – Artificial Intelligence, D – Operating Systems	Day 1
Slot2	B – Database Systems, E – Software Engineering	Day 2
Slot3	C – Computer Networks	Day 3

Assignment: A = Slot1, B = Slot2, C = Slot3, D = Slot1, E = Slot2 – identical to the plain-backtracking result and verified against all 7 constraints in Part (d.8).

Metric	Value
Assignment attempts	5
Rejected attempts	0
Backtracks	0
Domain values pruned	7

Part (f) – Comparative Analysis

f.1 Quantitative comparison

Metric	Plain Backtracking	Backtracking + Forward Checking	Difference
Assignment attempts	9	5	4 fewer (↓ 44.4%)
Rejected (failed) attempts	4	0	4 fewer
Accepted attempts (= number of variables)	5	5	Same
Backtracks (undoing earlier assignment)	0	0	Same for this instance
Domain values pruned (extra work)	0	7	7 more
Solution found	A=S1, B=S2, C=S3, D=S1, E=S2	A=S1, B=S2, C=S3, D=S1, E=S2	Identical

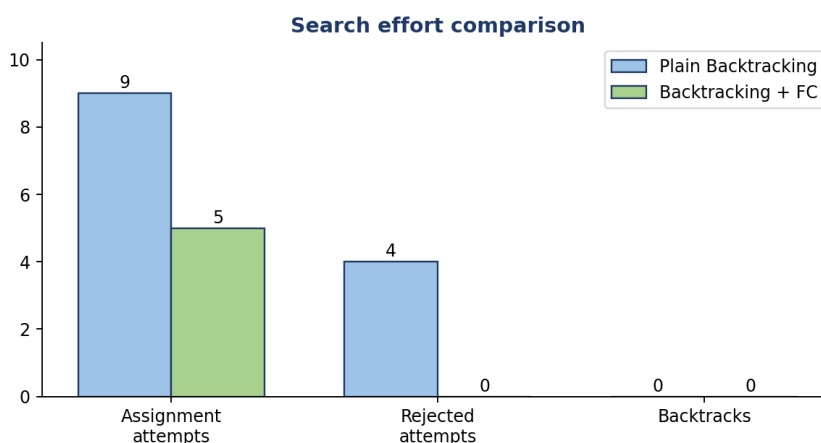


Figure 7: Search effort of the two algorithms on the exam-scheduling CSP.

f.2 Forward checking save work

Plain-search attempt that was wasted	Forward-checking step that removed it
Step 2: B = Slot1 (clashes with A)	Slot1 pruned from B after A = Slot1
Step 4: C = Slot1 (clashes with A)	Slot1 pruned from C after A = Slot1
Step 5: C = Slot2 (clashes with B)	Slot2 pruned from C after B = Slot2
Step 8: E = Slot1 (clashes with D)	Slot1 pruned from E after D = Slot1

Exactly the four rejected attempts in plain backtracking ($9 - 5 = 4$) are the ones forward checking eliminates, which explains the entire difference in attempts.

f.3 Comparison of efficiency

- **Number of assignment attempts:** FC needs 5 attempts, one per variable, which is the minimum possible for 5 variables. Plain backtracking needs 9, i.e. 80% more attempts than necessary.
- **Number of backtracks:** both algorithms needed 0 backtracks here, because with this variable/value order the first legal choice at every level leads to a solution. This is a property of this particular instance and ordering, not an advantage of either algorithm.
- **Overall efficiency:** FC does less search (fewer nodes in the tree) but performs 7 pruning operations. Since each pruning is a cheap domain deletion, and each wasted attempt in plain backtracking also costs a consistency test, FC is more efficient overall for this problem, but the gain is modest because the problem is tiny.
- **Solution quality:** both return the same valid timetable; FC does not change correctness, only effort.

f.4 Checking reduces the search effort

1. It prevents known-bad choices. A value that conflicts with an assigned neighbour is removed in advance, so the algorithm never spends an attempt on it (e.g. C's domain shrinks to {Slot3} before C is reached).
2. It detects dead ends early. If pruning empties some future variable's domain, the current assignment is rejected right away. Plain backtracking would only discover the dead end much later, after assigning further variables, and could waste a whole subtree.
3. It propagates information forward. Each assignment passes its consequence to all unassigned neighbours immediately, so later variables work with smaller, already-consistent domains.
4. It keeps the tree narrow. The branching factor falls from 3 to 2, 1, 1, 1 along the path (domain sizes at selection time: A=3, B=2, C=1, D=1, E=1), whereas plain backtracking always considers all 3 values at each level.

f.5 Limitations and wider perspective

- Forward checking only checks neighbours of the variable just assigned; it does not detect conflicts between two unassigned variables (that needs arc consistency / AC-3).
- On this small CSP the worst case of plain backtracking is bounded by $3^5 = 243$ complete assignments, but only 9 attempts were used. The benefit of FC grows quickly with more variables, tighter constraints and orderings that lead to dead ends.
- Combining FC with the MRV (choose the variable with the fewest remaining values, here C first) and degree heuristics would reduce search effort further, although the question fixes a static order.

f.6 Conclusion

The constraint-based examination scheduling problem demonstrates how Artificial Intelligence techniques can be used to solve real-world scheduling problems efficiently and systematically. In this problem, five university courses—Artificial Intelligence (A), Database Systems (B), Computer Networks (C), Operating Systems (D), and Software Engineering (E)—must be scheduled across three examination slots while satisfying all the specified conflict constraints. The problem was successfully modeled as a Constraint Satisfaction Problem (CSP) by defining the courses as variables, the three examination slots as their domains, and the student-enrollment conflicts as constraints.

Appendix – Python Implementation and Output

A.1 Source code (csp_solver.py)

The program below implements both algorithms with the same variable order and value order used in this report, and counts attempts, rejections and backtracks.

```
"""Exam-scheduling CSP: plain Backtracking vs Backtracking + Forward Checking."""
import copy

VARIABLES = ["A", "B", "C", "D", "E"]          # static order A->B->C->D->E
DOMAIN    = ["Slot1", "Slot2", "Slot3"]       # value order Slot1->Slot2->Slot3
EDGES     = [("A","B"),("A","C"),("B","C"),("B","D"),("C","D"),("C","E"),("D","E")]

NEIGHBOURS = {v: set() for v in VARIABLES}
for u, v in EDGES:
    NEIGHBOURS[u].add(v); NEIGHBOURS[v].add(u)

def consistent(var, val, assignment):
    """Value is legal if no already-assigned neighbour uses the same slot."""
    return all(assignment.get(n) != val for n in NEIGHBOURS[var])

# ----- plain BT
def backtracking(assignment, stats, log):
    if len(assignment) == len(VARIABLES):
        return assignment
    var = VARIABLES[len(assignment)]          # static variable order
    for val in DOMAIN:                        # static value order
        stats["attempts"] += 1
        if consistent(var, val, assignment):
            log.append((stats["attempts"], var, val, "ACCEPTED"))
            assignment[var] = val
            result = backtracking(assignment, stats, log)
            if result: return result
            del assignment[var]                # undo -> BACKTRACK
            stats["backtracks"] += 1
            log.append((stats["attempts"], var, val, "BACKTRACK"))
        else:
            stats["failed"] += 1
            log.append((stats["attempts"], var, val, "REJECTED (conflict)"))
    return None

# ----- BT + forward check
def forward_checking(assignment, domains, stats, log):
    if len(assignment) == len(VARIABLES):
        return assignment
    var = VARIABLES[len(assignment)]
    for val in list(domains[var]):            # only surviving values
        stats["attempts"] += 1
        new_domains = copy.deepcopy(domains)
        new_domains[var] = [val]
        pruned, wipeout = [], False
        for n in NEIGHBOURS[var]:
```

```

        if n not in assignment and val in new_domains[n]:
            new_domains[n].remove(val)          # forward-check pruning
            pruned.append((n, val))
            if not new_domains[n]: wipeout = True
    if wipeout:
        stats["failed"] += 1
        log.append((stats["attempts"], var, val, "WIPE-OUT", pruned, new_domains))
        continue
    assignment[var] = val
    log.append((stats["attempts"], var, val, "ACCEPTED", pruned, new_domains))
    result = forward_checking(assignment, new_domains, stats, log)
    if result: return result
    del assignment[var]
    stats["backtracks"] += 1
return None

if __name__ == "__main__":
    s1 = {"attempts":0,"failed":0,"backtracks":0}; l1 = []
    sol1 = backtracking({}, s1, l1)
    print("PLAIN BACKTRACKING"); [print(" ", r) for r in l1]
    print(" solution:", sol1, s1)

    s2 = {"attempts":0,"failed":0,"backtracks":0}; l2 = []
    init = {v: list(DOMAIN) for v in VARIABLES}
    sol2 = forward_checking({}, init, s2, l2)
    print("\nFORWARD CHECKING")
    for r in l2: print(" ", r[:5]); print(" domains:", r[5])
    print(" solution:", sol2, s2)

```

A.2 Program output

The output matches the hand-worked tables in Parts (d) and (e): 9 attempts / 4 rejected / 0 backtracks for plain backtracking and 5 attempts / 0 rejected / 0 backtracks for forward checking.

```
IDLE Shell 3.13.11
File Edit Shell Debug Options Window Help
Python 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Nadesa Krishnan/OneDrive/Documents/CO1-AT2.py =====
PLAIN BACKTRACKING
=====
(1, 'A', 'Slot1', 'ACCEPTED')
(2, 'B', 'Slot1', 'REJECTED (conflict)')
(3, 'B', 'Slot2', 'ACCEPTED')
(4, 'C', 'Slot1', 'REJECTED (conflict)')
(5, 'C', 'Slot2', 'REJECTED (conflict)')
(6, 'C', 'Slot3', 'ACCEPTED')
(7, 'D', 'Slot1', 'ACCEPTED')
(8, 'E', 'Slot1', 'REJECTED (conflict)')
(9, 'E', 'Slot2', 'ACCEPTED')

Solution:
{'A': 'Slot1', 'B': 'Slot2', 'C': 'Slot3', 'D': 'Slot1', 'E': 'Slot2'}

Statistics:
Assignment Attempts : 9
Failed Attempts      : 4
Backtracks           : 0

=====
BACKTRACKING + FORWARD CHECKING
=====

Attempt: 1
Variable: A
Value: Slot1
Status: ACCEPTED
Pruned: [('C', 'Slot1'), ('B', 'Slot1')]
Domains:
  A -> ['Slot1']
  B -> ['Slot2', 'Slot3']
  C -> ['Slot2', 'Slot3']
  D -> ['Slot1', 'Slot2', 'Slot3']
  E -> ['Slot1', 'Slot2', 'Slot3']

Attempt: 2
Variable: B
Value: Slot2
Status: ACCEPTED
Pruned: [('D', 'Slot2'), ('C', 'Slot2')]
Domains:
  A -> ['Slot1']
  B -> ['Slot2']
  C -> ['Slot3']
  D -> ['Slot1', 'Slot3']
  E -> ['Slot1', 'Slot2', 'Slot3']

Attempt: 3
Variable: C
Value: Slot3
Status: ACCEPTED
Pruned: [('E', 'Slot3'), ('D', 'Slot3')]
Domains:
  A -> ['Slot1']
  B -> ['Slot2']
  C -> ['Slot3']
  D -> ['Slot1']
  E -> ['Slot1', 'Slot2']

Attempt: 4
Variable: D
Value: Slot1
Status: ACCEPTED
Pruned: [('E', 'Slot1')]
Domains:
  A -> ['Slot1']
  B -> ['Slot2']
  C -> ['Slot3']
  D -> ['Slot1']
  E -> ['Slot2']

Attempt: 5
Variable: E
Value: Slot2
Status: ACCEPTED
Pruned: []
Domains:
  A -> ['Slot1']
  B -> ['Slot2']
  C -> ['Slot3']
  D -> ['Slot1']
  E -> ['Slot2']

Solution:
{'A': 'Slot1', 'B': 'Slot2', 'C': 'Slot3', 'D': 'Slot1', 'E': 'Slot2'}

Statistics:
Assignment Attempts : 5
Failed Attempts      : 0
Backtracks           : 0
>>>
```